NEXT GEN SOFTWARE HUB PVT. LTD. · PATNA

Internship Report

Web Development Internship · Weeks 01–08

INTERN	Toheed Mehmood
LOCATION	Lahore, Pakistan · remote
PROGRAMME	Eight weeks · five projects
STACK	HTML5, CSS3, vanilla JavaScript. No framework, no build step.
SUBMITTED	9 November 2026

Prepared for the Week 7–8 final review. The five project folders, their READMEs and the submission checklist accompany this report.

Next Gen Software Hub Pvt. Ltd., Patna Programme: Web Development Internship Duration: eight weeks (Weeks 01–08) Deliverables: five projects

---

The deal

Five projects. Eight weeks. HTML, CSS, JavaScript, no framework, no build step, no bundler. Every deliverable had to open by double-clicking `index.html`. Google Fonts was the only thing I was allowed to pull from the network.

People assume the hard part of that setup is writing more code. It isn't. The hard part is that nothing catches your mistakes for you. No compiler, no framework runtime, no bundler warning. A broken selector just does nothing. A bad state update paints the wrong screen. You find out by using the page, which is why most of this report ends up being about testing.

First fortnight: a page about me, then a list that fights back

Portfolio came first, and I assumed it would be a warm-up. The layout was. The accessibility wasn't. Sticky nav, scroll-spy, `clamp()` type scale, animated skill bars, project grid; that part came together in an afternoon. Then I started tabbing through it and found problems in every section. Focus disappearing into the background. A nav that made sense with a mouse and not with a keyboard. I rebuilt the navigation three times before the order felt right.

The JavaScript is deliberately thin. `IntersectionObserver` handles the reveal animations, and one delegated listener covers the contact form instead of a handler on every field. The CV lives in its own HTML file, linked from the hero, so it opens immediately instead of triggering a download.

Taskbook was next. Add a task, tick it, edit, delete. The CRUD was routine. Delete was not. I did not want a destructive action with no way back, so deleted tasks move to Trash and a toast offers undo for six seconds. The undo handler closes over a snapshot taken at delete time, which means it restores the correct task even if you changed three others in the meantime. That trick reappears in TaskFlow.

Search is debounced at 200 ms. Bulk actions pause for a beat before they mutate, otherwise the disabled state flashes for a single frame and nobody registers it.

Plates: sixteen drawings, one blocked API

Sixteen SVG plates, drawn locally, arranged in a gallery with category filters, debounced search, three density modes and a lightbox. SVG was the practical choice: no separate image files, no blurring when the grid scales, and the whole gallery stays small.

The animation work was the part I enjoyed. Filtering reorders the grid, so I measure each tile before and after the DOM change and animate the difference with FLIP. Skip that step and the tiles snap to their new positions. Do it properly and the browser composites the movement instead of recalculating layout on every frame. I had read about FLIP a few times. This was the first project where it solved a real problem.

The other problem was `file://`. The catalogue lives in `assets/works/index.json` and the gallery fetches it. Served over HTTP, no issue. Opened from disk, Chrome blocks the request and the console fills with red. Ignoring that would have left the submission with a broken gallery, which seemed like a bad idea given the brief. So the fetch became an enhancement rather than a requirement: JSON when available, an inline copy of the same records when not. Both paths get tested.

Quiz Arena: six screens, one switch

Twenty questions across four difficulty levels and seven topics. Runs of five, ten, fifteen or the full twenty. Thirty seconds per question. Pause overlay, results, local leaderboard, fault screen.

The screens were the first design decision. Six booleans would have been six chances to show the wrong combination. Instead there is a single phase value. Set it to `quiz` and the quiz screen appears; set it to `results` and the results screen takes over. There is no reachable state where both are visible, because the code never asks for one.

Validation surprised me with how much room it needed. The setup screen counts the questions that match the chosen filters and refuses to start a ten-question run when only six qualify. Silently shortening the run would have been easier to build and worse to use. Errors are announced through `role="alert"`.

Announcements were broken for a while. A local variable called `announce` shadowed the helper that writes to the live region, so nothing was ever read out. Nothing in DevTools looked wrong. Renaming the variable fixed it. The remote question bank is skipped on `file://` as well, because an error toast about a fetch you cannot influence is just noise.

Scores live in a local top-ten table. Export, import, clear after confirmation. Import validates the schema before touching anything. A JSON file is not trustworthy just because it ends in `.json`.

TaskFlow: the board, and the bug I remember

Four columns, twenty-one cards, three views, undo/redo, search, filters, insights, settings, offline storage. Every card is labelled sample content. It demonstrates the board; it is not a record of my internship.

The architecture mattered more here than anywhere else. One pure reducer owns the state. Changes leave the interface as action objects through `dispatch`. Subscribers get notified, the view redraws, handlers never touch the DOM, and `localStorage` is off limits outside the store.

That arrangement made the later features cheap. History is a stack next to the reducer. Saving is a flush after each commit. The simulated sync simply dispatches ordinary actions when its timer finishes.

Replay produced the bug I remember best. Moving an entry between the undo and redo stacks has to happen before the replay, not after. Get the order wrong and the buttons calculate from a history that has already moved on. Undo stays enabled with an empty stack. Finding that took far longer than fixing it.

Keyboard support was the largest single piece of work. Roving `tabindex` so one card per column sits in the tab sequence, arrow keys to move, Home and End for the edges, dialogs that trap focus and return it to the trigger, a keyboard path for drag and drop, and live regions for sync and undo.

What broke, in order

Quiz Arena shipped a dead start button. The console said `TypeError: Cannot create property 'hidden' on boolean 'true'`. A helper that toggled screens by selector was treating its map values as DOM nodes when they were booleans. It threw during the first render, so initialisation stopped and the page did nothing at all. One line fixed it. The lesson was not about syntax: you cannot read your way to a runtime throw. After that I tested through the interface.

Taskbook and Plates were still using `var`, which contradicts the design system. I scripted the rewrite to `let`, applied it, ran `node --check`, and repeated the browser pass over each feature.

The OneDrive folder fought back. Some files are placeholders, and the shell would not write, overwrite, move or delete them in place. Everything went through the editor instead, with backups kept outside the synced tree. Not an application bug, but it shaped the rest of the eight weeks.

Three failures turned out to be bad tests. A search check deleted the task it was about to look for. A tile count was read before an async render settled. A quiz filter looked ignored until I noticed that every chip starts checked, meaning include everything. Two of them needed no code change. Confirming that first was the point.

Several READMEs and file headers described features the code did not have: arrow functions, destructuring, spread, optional chaining. I counted the constructs file by file and rewrote the claims. Early drafts of this report had the same problem. They described the intention, not the build.

Portfolio had six dead links, two cards for projects that were not part of the submission, a description of a four-column board as three-column, and a quiz card advertising a fixed ten-question run. I checked every card against the running app and wrote a small Node script that walks each `href` and `src` to confirm the target exists. The dead links were not a mystery. I simply had not checked.

My first PDF export carried Chrome's date and URL across every margin. The HTML looked clean; the PDF did not. I regenerated it with the header disabled and read the extracted text before signing off.

What I actually learned

Accessibility stopped being a final checklist. Landmarks, one `h1`, `fieldset` and `legend`, native validation first, `aria` only when the markup cannot carry the meaning. It changed how the pages were built, not just how they were annotated.

CSS became a control problem rather than a property problem. Custom properties, a dark-first token set, `clamp()` typography, Grid and Flexbox, `:focus-visible`, reduced motion, FLIP, borders instead of shadows. Shared tokens kept five projects recognisably related without making them copies.

JavaScript changed the most. In week one I changed the DOM wherever it was convenient. By week eight one state object per app produced the view. Event delegation replaced rows of listeners. Closures handled debounce timers, private caches and undo snapshots. `Promise` and `async` / `await` appeared where work was genuinely asynchronous. TaskFlow ended with reducer, `dispatch` and `subscribe`.

Browser APIs came with `file://` caveats I did not expect: `localStorage` with a schema version, `FileReader`, `Blob`, `IntersectionObserver`, `matchMedia`, `history`, `Intl.DateTimeFormat`, the File API. A couple of evenings went into learning that `file://` is not a local server.

Testing is the change I value most. Schema checks on imports, `textContent` instead of `innerHTML`, `try` / `catch` around storage with a sample fallback, keyboard parity for pointer actions, headless Chrome with a virtual time budget, and a link checker instead of hoping.

Final position

Five projects. All open from disk. All documented. One design system, one webfont dependency. I clicked through each in Chrome, watched the console and checked every local reference.

The structure moved further than the visuals. Week one was DOM-first. By the last week, state came first. Once that clicked, undo/redo, persistence and sync stopped being separate features and became side effects of the same design.

Testing moved too. The Quiz Arena crash and the portfolio links looked fine in the source and failed within seconds in a browser. I no longer trust code because it reads well.

If I started again

I would work outside the synced folder. That single change would have saved a week of friction.

I would write interaction tests first. Every failure above is the kind a small suite catches: a throw during the first render, a path that points nowhere, a count taken before the re-render lands.

And I would keep documentation honest while building. Claims drift away from code faster than expected.

Thanks to the Next Gen Software Hub team for the brief and for the time to work through it.